

If you're aiming for an **A-grade final-year robotics project**, don't just use a pre-trained "fall detection" model and call it a day. Your examiners will want to see that you understand the complete pipeline and have done engineering work beyond downloading a model.

Recommended Architecture

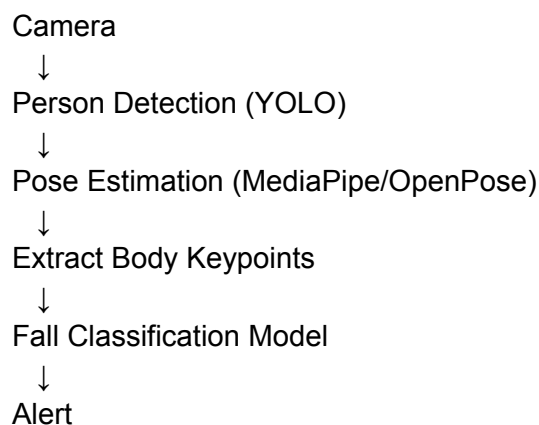
Option 1 (Best for Your Project)

Camera → Person Detection → Pose Estimation → Fall Classification

This is much better than training a CNN directly on images because:

- Works with smaller datasets
- More explainable
- Faster on Raspberry Pi
- Easier to achieve high accuracy
- Looks more advanced in a viva

Pipeline:



Step 1: Learn the Basics

Before touching datasets:

OpenCV

Learn:

- Image reading
- Video capture
- Bounding boxes

- Drawing landmarks

Example:

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    cv2.imshow("Camera", frame)

    if cv2.waitKey(1) == 27:
        break
```

Step 2: Obtain a Fall Dataset

Look at:

UR Fall Detection Dataset

Contains:

- Falls
- Walking
- Sitting
- Daily activities

or

Le2i Fall Detection Dataset

Widely used in research.

or

UP-Fall Dataset

Includes:

- Falls
- Normal activities
- Multiple viewpoints

For a project, **UP-Fall** is a strong choice.

Step 3: Detect the Person

Use YOLOv8.

Install:

```
pip install ultralytics
```

Example:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

```
results = model(frame)
```

This gives you:

- Person location
- Bounding box

Step 4: Extract Human Pose

Use MediaPipe.

```
pip install mediapipe
```

You'll obtain:

- Head
- Shoulders
- Hips
- Knees
- Ankles

33 body landmarks.

Example:

```
pose.process(frame)
```

Output:

x,y positions of joints

Step 5: Create Your Own Features

This is where you earn marks.

Instead of using images directly, compute:

Body Angle

Standing:

|
|
|

Falling:

Calculate:

$\text{angle} = \arctan(\dots)$

Height-to-Width Ratio

Standing:

$\text{height} \gg \text{width}$

Fallen:

$\text{width} > \text{height}$

Feature:

ratio = height / width

Hip Velocity

Fall:

sudden downward motion

Track:

hip_y(t)

Compute velocity.

Head Position

Sudden drop:

head height decreases rapidly

Step 6: Train Your Own Classifier

Create labels:

0 = Normal

1 = Fall

Features:

Body angle

Bounding box ratio

Hip speed

Head speed

Train:

Random Forest

or

XGBoost

or

SVM

Example:

```
from sklearn.ensemble import RandomForestClassifier
```

These models work surprisingly well.

Step 7: Improve to A-Grade Level

Most students stop here.

You can go further.

Temporal Detection

Instead of analyzing one frame:

Use:

Previous 30 frames

Model:

LSTM

Input:

pose(t-30 ... t)

Output:

Fall / No Fall

This resembles modern research.

Step 8: False Alarm Reduction

Many systems mistake:

- Lying down
- Sitting
- Bending

for falls.

Add:

Fall detected



Person remains horizontal?



No movement for 5 seconds?



Confirm Fall

This greatly improves reliability.

Step 9: Raspberry Pi Deployment

Train on your laptop.

Deploy on Pi:

YOLOv8n

+

MediaPipe

+

Random Forest

This is realistic on a Raspberry Pi 4.

Avoid training neural networks on the Pi.

What Would Impress an Examiner?

Instead of saying:

"I used a fall detection model."

Say:

"A custom vision-based fall detection pipeline was developed using YOLOv8 for human localization, MediaPipe pose estimation for skeletal keypoint extraction, and a Random Forest classifier trained on temporal posture features derived from the UP-Fall dataset."

That sounds like a genuine robotics and AI engineering project.

For your current scope (13-week project, Raspberry Pi 4, companion robot, 3rd-year robotics student), I would recommend:

YOLOv8n + MediaPipe Pose + Custom Random Forest Fall Classifier + 5-second confirmation logic

This is achievable, research-oriented, and strong enough for an A-grade project if implemented and evaluated properly.